FORMATION « PYTHON EN 24 HEURES » | SÉANCE 01

Penser comme une machine : algorithmique et conditions

Chapitres 1 et 2 du Tome 1

1 h de cours (présentation Beamer) • 1 h de pratique (notebook Jupyter)

HOUNKPE Mawulikplimi Roland et DJABON Ounimborbitibou

Objectifs de la séance

- Définir ce qu'est un algorithme avec des exemples de la vie courante
- Décrire un traitement en pseudo-code, avant d'écrire la moindre ligne de code
- Utiliser les structures conditionnelles SI, SINON et SINON SI
- Assembler des conditions avec les opérateurs de comparaison et les connecteurs ET, OU, NON
- Préparer le TP de 1 h : cinq exercices progressifs avec corrigés

Avant de commencer : ce dont vous avez besoin

Cette première séance ne suppose aucune connaissance en programmation. Trois dispositions suffisent pour bien démarrer.

- Un ordinateur avec Python 3 installé pour la partie TP
- De la patience : on réfléchit d'abord, on tape ensuite
- Le Tome 1 du cours sous la main pour approfondir après la séance
- Un cahier pour garder trace de vos essais et de vos erreurs

Qu'est-ce qu'un algorithme ?

- Une **suite finie d'instructions** non ambiguës qui transforme des données de départ en résultat
- Exemple quotidien : une recette de cuisine (ingrédients en entrée, plat en sortie, étapes ordonnées)
- Exemple quotidien : monter un meuble avec une notice, faire la lessive, calculer une moyenne
- Un ordinateur exécute très vite, mais **exactement** ce qu'on lui dit, ni plus ni moins
- Bien programmer, c'est d'abord bien **décrire** ce que l'on veut obtenir

Du problème au programme : trois temps

1. Analyser

- Quelles sont les **entrées** ?
- Quel est le **résultat** attendu ?
- Quelles règles de calcul s'appliquent ?

2. Décrire

- Écrire les étapes en **pseudo-code**
- Le pseudo-code est du français structuré
- Aucune syntaxe de langage à retenir

3. Tester

- Exécuter « à la main » sur un exemple
- Vérifier les cas limites : zéro, négatif, vide
- Corriger puis traduire en Python

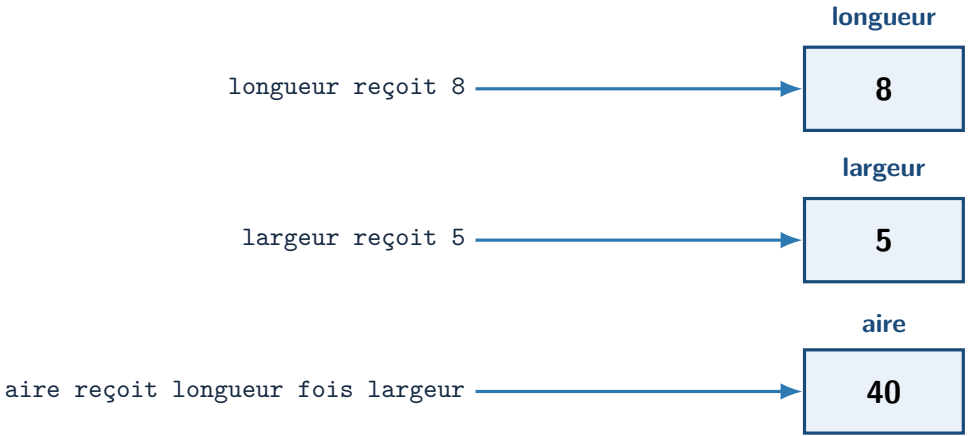
Le pseudo-code : variables et affectation

- Une **variable** est une boîte étiquetée qui garde une valeur en mémoire
- L'affectation **reçoit** place une valeur dans la boîte

```
ALGO AireDuRectangle
longueur reçoit 8
largeur reçoit 5
aire reçoit longueur fois largeur
AFFICHER "Aire : ", aire
```

L'affectation n'est pas une égalité mathématique : c'est un rangement.

Schéma : la variable, une boîte étiquetée



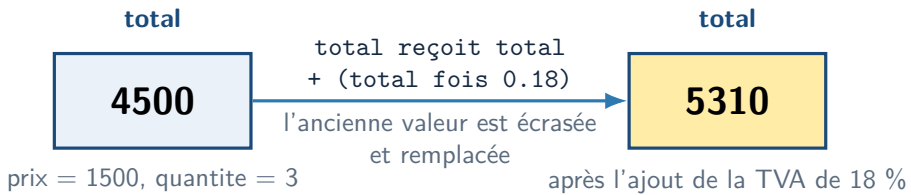
À gauche, l'instruction du pseudo-code ; à droite, la boîte qui reçoit la valeur. L'étiquette ne change jamais : elle sert à retrouver la boîte.

Entrées, sorties et calculs

- **ENTRER** lit une donnée fournie par l'utilisateur
- **AFFICHER** montre un résultat à l'écran
- Les calculs suivent les priorités habituelles : fois et divisé avant plus et moins
- Les parenthèses rendent l'ordre explicite

```
ALGO PrixTotal
ENTRER prix
ENTRER quantite
total reçoit prix fois quantite
total reçoit total plus (total fois 0.18)
AFFICHER "À payer : ", total
```

Schéma : mettre à jour une variable



Règle d'or : une affectation écrase toujours l'ancienne valeur. La boîte « total » ne garde qu'une seule valeur à la fois.

Les conditions : prendre des décisions

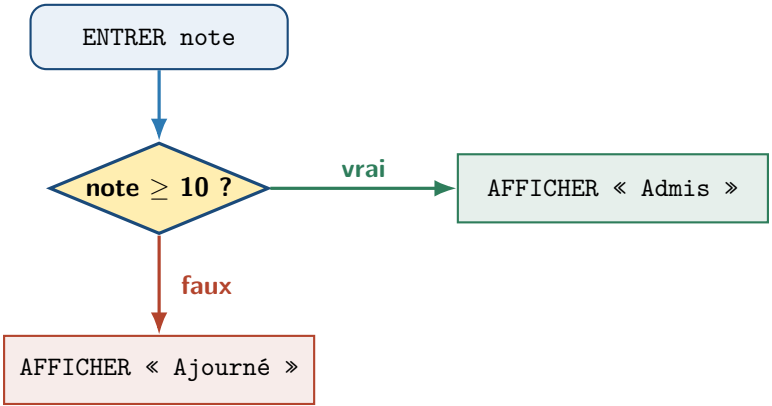
- Sans conditions, un programme fait toujours la même chose
- La structure **SI ... ALORS ... SINON** choisit un chemin selon un test
- Le test produit toujours l'une des deux valeurs : **vrai** ou **faux**
- Un seul des deux chemins est exécuté, jamais les deux

Structure SI / SINON

```
ALGO AdmisOuNon
ENTRER note
SI note >= 10 ALORS
    AFFICHER "Admis"
SINON
    AFFICHER "Ajourné"
FIN SI
```

L'indentation (le décalage) montre ce qui appartient au SI et ce qui appartient au SINON.

Schéma : le chemin d'une condition



Le losange évalue le test : il produit vrai ou faux, puis une seule flèche est suivie, jamais les deux.

Les opérateurs de test

Opérateur	Signification	Exemple avec x = 7
=	égal à	x = 7 est vrai
!=	différent de	x != 3 est vrai
<	strictement inférieur	x < 7 est faux
<=	inférieur ou égal	x <= 7 est vrai
>	strictement supérieur	x > 0 est vrai
>=	supérieur ou égal	x >= 10 est faux

Connecteurs : ET (les deux tests vrais), OU (au moins un vrai), NON (inverse le résultat).

Conditions imbriquées et SINON SI

```
ALGO MentionSelonNote
ENTRER note
SI note >= 16 ALORS
    AFFICHER "Très bien"
SINON SI note >= 14 ALORS
    AFFICHER "Bien"
SINON SI note >= 10 ALORS
    AFFICHER "Passable"
SINON
    AFFICHER "Insuffisant"
FIN SI
```

Les tests se lisent de haut en bas : le premier qui est vrai gagne, les autres sont ignorés.

Assembler des conditions avec ET, OU

```
ALGO AccesAutorise
ENTRER age
ENTRER carteValide
SI age >= 18 ET carteValide = vrai ALORS
    AFFICHER "Accès autorisé"
SINON
    AFFICHER "Accès refusé"
FIN SI
```

Il n'y a aucun besoin de tester age deux fois : un seul SI bien écrit suffit.

Un premier pont vers Python

- SI se traduit par **if**, SINON par **else**
- Les deux points ouvrent un bloc
- L'**indentation** de 4 espaces remplace le FIN SI
- Vous verrez cela en détail à la séance 04

```
note = 13
if note >= 10:
    print("Admis")
else:
    print("Ajourné")
```

Attention

Les pièges du débutant

Confondre le test d'égalité (= en pseudo-code, == en Python) avec l'affectation. Oublier le cas SINON alors qu'il faut une réponse dans tous les cas. Écrire des conditions dans un ordre tel qu'un test trop général masque les tests suivants. Vérifiez toujours votre algorithme à la main sur deux ou trois exemples.

À retenir

L'essentiel de la séance 01

Un algorithme est une suite d'instructions finie et non ambiguë. On analyse, puis on décrit en pseudo-code, puis on teste à la main avant de coder. Les structures SI, SINON et SINON SI permettent de prendre des décisions ; les connecteurs ET, OU, NON les combinent. Cette discipline de pensée vaut pour tous les langages.

ditions

Dans ce premier TP, vous écrirez d'abord des algorithmes en pseudo-code sur papier, puis vous les traduirez en Python directement dans le notebook. L'objectif est de verrouiller la logique de décision.

- **Exercice 1 (Basique)** : Périmètre et aire d'un rectangle
- **Exercice 2 (Basique)** : Pair ou impair
- **Exercice 3 (Intermédiaire)** : Trier trois nombres sans sorted()
- **Exercice 4 (Intermédiaire)** : Calculateur de mention
- **Exercice 5 (Défi)** : Année bissextile

Ouvrez le notebook « Seance_01_penser_comme_une_machine.ipynb » pour commencer.

Pour la prochaine séance

Conseil de travail

Séance 02 : les boucles. Nous apprendrons à répéter des instructions proprement avec TANT QUE et POUR, à compter, à accumuler et à écrire nos premiers vrais algorithmes. Relisez les opérateurs de test du Tome 1, ils serviront partout.